

Architecture & Design Document: The Redemption

Role: Lead Cloud Engineer Assessment

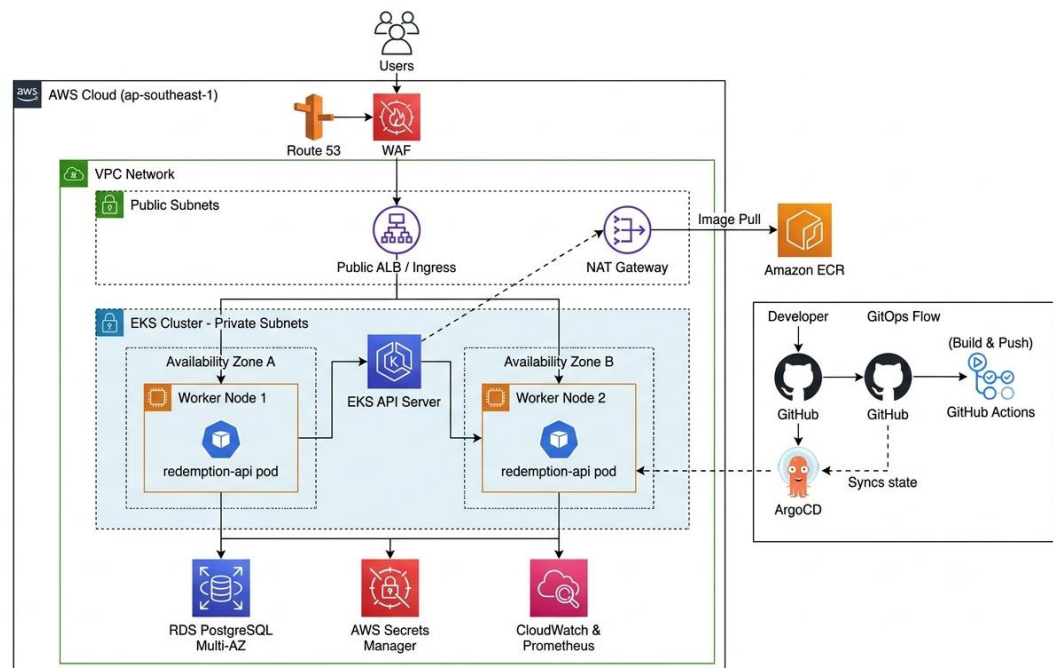
Executive Summary & Architecture

The *Redemption* microservice operates on a highly available, secure, and containerized AWS Elastic Kubernetes Service (EKS) infrastructure. Designed for zero-downtime deployments and extreme load scalability, the architecture utilizes Multi-AZ compute nodes and a Multi-AZ RDS PostgreSQL database to ensure strict fault tolerance.

To minimize the blast radius and ensure modularity, the foundational infrastructure was provisioned using a layered Infrastructure-as-Code (IaC) strategy via **Terraform**. By segmenting the state files (Network, Infrastructure, EKS, Database, and Helm Add-ons), we ensure safe, independent updates to distinct architectural layers.

Furthermore, to eliminate configuration drift and minimize day-two operational toil, the platform utilizes **ArgoCD** as a strict GitOps controller. This architecture ensures that the GitHub repository remains the single immutable source of truth for all Kubernetes application states, actively rejecting unauthorized manual cluster modifications.

Architecture Diagram



Scalability Strategy

Overview of the Traffic Profile

The primary operational challenge for The Redemption microservice is managing a volatile traffic profile characterized by a steady baseline punctuated by sudden, extreme 10x spikes driven by global Flash Sales. Given that service failure results in direct revenue loss, the scalability architecture must operate completely without manual intervention to guarantee zero downtime. To mitigate this risk, a dual-layer automated scaling strategy is implemented, separating application-level concurrency from underlying infrastructure compute capacity.

Application-Layer Scaling: Horizontal Pod Autoscaler (HPA)

Horizontal scaling at the container level is driven by the Kubernetes Horizontal Pod Autoscaler (HPA), which dynamically adjusts the replica count of the application deployment based on real-time resource constraints.

Baseline Configurations: The application deployment maintains a strict baseline of a minimum of 3 replicas to guarantee high availability across multiple availability zones during nominal traffic periods. The scale-out ceiling is set to a maximum of 50 replicas to absorb maximum load spikes during peak Flash Sale events.

Target Utilization Metrics: The HPA continuously monitors resource utilization via the Metrics Server add-on. The scaling metrics are bound to dual resource thresholds.

CPU Utilization: Triggered when average CPU utilization across the existing replica set exceeds 60%.

Memory Utilization: Triggered when average memory utilization across the existing replica set exceeds 70%.

Reconciliation and Traffic Routing: When either threshold is breached, the HPA dynamically instantiates additional redemption-api pods. The AWS Load Balancer Controller automatically registers the new pod endpoints, distributing incoming traffic evenly across the expanded footprint.

Infrastructure-Layer Scaling: Karpenter and Managed Node Groups

To ensure that container scaling is backed by adequate physical compute resources, the underlying Amazon EC2 infrastructure scales dynamically in tandem with the application layer.

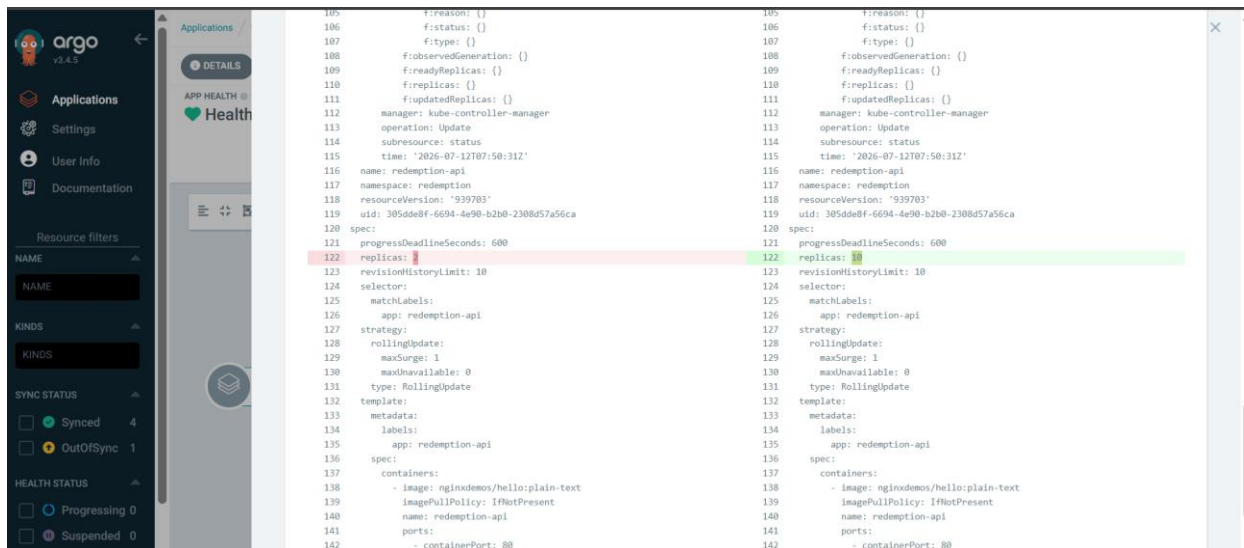
System Node Group Isolation: Core cluster control workloads, including the AWS Load Balancer Controller, Metrics Server, CoreDNS, and ArgoCD components, are isolated onto a dedicated system managed node group composed of t3.medium instances. This group is bounded between a minimum of 2 and a maximum of 5 instances, ensuring control plane stability is never compromised by application scaling events.

Application Node Group Scaling: The application workload layer utilizes a managed node group incorporating highly efficient m6i.large and m6a.large instances to process business logic. The group initiates with a baseline desired capacity of 3 nodes and can scale up to a maximum ceiling of 16 compute instances.

Dynamic Node Provisioning via Karpenter: In conjunction with managed node groups, Karpenter is deployed to eliminate the provisioning delays inherent in traditional Auto Scaling Groups. When the HPA generates pods that cannot be scheduled due to insufficient cluster resources, Karpenter evaluates the pending pod requirements and directly communicates with the Amazon EC2 API to provision appropriately sized nodes within seconds. This ensures rapid infrastructure expansion during rapid traffic onset.

Scalability Validation and Evidence

To verify the efficacy of the automated dual-layer scaling mechanism under maximum load constraints, a rigorous synthetic spike test was executed utilizing a localized k6 load-testing script. The test subjected the system to a rapid ramp-up simulating heavy concurrent user traffic arriving at the application endpoint.



```
105 f:reason: {}
106 f:status: {}
107 f:type: {}
108 f:observedGeneration: {}
109 f:readyReplicas: {}
110 f:replicas: {}
111 f:updateReplicas: {}
112 manager: kube-controller-manager
113 operation: Update
114 subresource: status
115 time: '2026-07-12T07:50:31Z'
116 name: redemption-api
117 namespace: redemption
118 resourceVersion: '939703'
119 uid: 305dde8f-6694-4e90-b2b0-2308d57a56ca
120 spec:
121   progressDeadlineSeconds: 600
122   replicas: 10
123   revisionHistoryLimit: 10
124   selector:
125     matchLabels:
126       app: redemption-api
127   strategy:
128     rollingUpdate:
129       maxSurge: 1
130       maxUnavailable: 0
131     type: RollingUpdate
132   template:
133     metadata:
134       labels:
135         app: redemption-api
136     spec:
137       containers:
138         - image: nginxdemo/hello:plain-text
139           imagePullPolicy: IfNotPresent
140           name: redemption-api
141           ports:
142             - containerPort: 80
```

ArgoCD detecting dynamic replica scaling triggered by the EKS Horizontal Pod Autoscaler during a simulated 10x traffic event.

Reliability & Security

Defense in Depth Architecture

The Redemption microservice employs a strict "Defense in Depth" strategy, ensuring that security and operational resilience are not reliant on a single perimeter. Instead, safeguards are layered throughout the network, compute, and application tiers. This approach minimizes the potential blast radius of a security event and guarantees high availability during localized infrastructure degradation.

Network Isolation via Private Worker Nodes

To protect the compute layer from external threats, all EKS worker nodes are strictly isolated within private subnets and have no direct ingress path from the public internet. External user traffic must pass through AWS WAF (Web Application Firewall) for malicious request filtering before reaching the Public Application Load Balancer (ALB). The ALB then acts as a secure proxy, routing legitimate traffic across the network boundary to the isolated worker nodes. Outbound traffic is similarly restricted, passing exclusively through a designated NAT Gateway.

Fault Tolerance via Multi-AZ Deployments

To guarantee zero downtime in the event of an AWS data center impairment, the architecture is highly available by default, spanning two geographically isolated Availability Zones (AZ-A and AZ-B).

Compute Tier: The EKS managed node groups distribute worker nodes across both AZs. The Kubernetes scheduler utilizes topology spread constraints to ensure redemption-api pod replicas are balanced across these distinct failure domains.

Data Tier: The Amazon RDS PostgreSQL database is deployed in a Multi-AZ configuration. It synchronously replicates transaction data to a hot standby instance in the secondary AZ, enabling automatic, seamless failover with zero data loss if the primary instance fails.

Principle of Least Privilege via IRSA

To secure AWS API access from within the Kubernetes cluster, the architecture utilizes IAM Roles for Service Accounts (IRSA) rather than attaching broad IAM policies to the underlying EC2 instances. By tightly coupling a specific IAM role to the redemption-api Kubernetes ServiceAccount, the application pods are granted only the precise, granular permissions required to retrieve database credentials from AWS Secrets Manager and emit metrics to CloudWatch. This strict adherence to the principle of least privilege prevents lateral privilege escalation within the cluster.

Reliability Validation and Evidence

To validate the cluster's fault tolerance, a worker node failure was artificially induced by forcefully draining a node of all its running workloads.

```
WaiYan@RGN24NB-dmcOVkp MINGW64 ~/Desktop/terraform/accor-redemption-eks-lab/my-gitops-infra (main)
$ kubectl get pods -n redemption
NAME                                READY   STATUS    RESTARTS   AGE
redemption-api-57cfff9748-jfp6z     1/1     Running   0           47s
redemption-api-57cfff9748-xvtttd    1/1     Running   0           47s

WaiYan@RGN24NB-dmcOVkp MINGW64 ~/Desktop/terraform/accor-redemption-eks-lab/my-gitops-infra (main)
$ kubectl delete pod -l app=redemption-api -n redemption
pod "redemption-api-57cfff9748-jfp6z" deleted
pod "redemption-api-57cfff9748-xvtttd" deleted

WaiYan@RGN24NB-dmcOVkp MINGW64 ~/Desktop/terraform/accor-redemption-eks-lab/my-gitops-infra (main)
$ kubectl get pods -n redemption -w
NAME                                READY   STATUS             RESTARTS   AGE
redemption-api-57cfff9748-7sh6s     1/1     Running            0           10s
redemption-api-57cfff9748-zxh9w     0/1     ContainerCreating   0           10s
redemption-api-57cfff9748-zxh9w     1/1     Running            0           33s

WaiYan@RGN24NB-dmcOVkp MINGW64 ~/Desktop/terraform/accor-redemption-eks-lab/my-gitops-infra (main)
$ kubectl get pods -n redemption -w
NAME                                READY   STATUS    RESTARTS   AGE
redemption-api-57cfff9748-7sh6s     1/1     Running   0           54s
redemption-api-57cfff9748-zxh9w     1/1     Running   0           54s
```

Above figure is Validation of Kubernetes self-healing. The deployment controller successfully identified a forced pod failure and provisioned replacements with zero downtime.

```
WaiYan@RGN24NB-dmcOVkp MINGW64 ~/Desktop/terraform/accor-redemption-eks-lab/my-gitops-infra (main)
$ kubectl drain ip-10-0-12-13.ap-southeast-1.compute.internal --ignore-daemon-
ydir-data
will retry after 5s): Cannot evict pod as it would violate the pod's disruption budget.
pod/metrics-server-84484689f8-qmrjxj evicted
evicting pod kube-system/coredns-586f9b4c8d-z28mj
pod/loki-gateway-5cd77fcb6b-v67qg evicted
error when evicting pods/"coredns-586f9b4c8d-z28mj" -n "kube-system" (will retry after 5s): Cannot evict pod as it would violate the pod's disruption budget.
pod/grafana-6b69464c79-vv2wd evicted
pod/redemption-api-57cfff9748-7sh6s evicted
pod/loki-0 evicted
pod/coredns-586f9b4c8d-pp5mm evicted
evicting pod kube-system/karpenter-677cb6cfb9-vdmtm
error when evicting pods/"karpenter-677cb6cfb9-vdmtm" -n "kube-system" (will retry after 5s): Cannot evict pod as it would violate the pod's disruption budget.
evicting pod kube-system/coredns-586f9b4c8d-z28mj
error when evicting pods/"coredns-586f9b4c8d-z28mj" -n "kube-system" (will retry after 5s): Cannot evict pod as it would violate the pod's disruption budget.
pod/ingress-nginx-controller-cf668668c-6zrhz evicted
evicting pod kube-system/karpenter-677cb6cfb9-vdmtm

WaiYan@RGN24NB-dmcOVkp MINGW64 ~/Desktop/terraform/accor-redemption-eks-lab/my-gitops-infra (main)
$ kubectl get pods -n redemption -w
NAME                                READY   STATUS             RESTARTS   AGE
redemption-api-57cfff9748-zxh9w     0/1     Terminating       0           5m48s
redemption-api-57cfff9748-zxh9w     0/1     Terminating       0           5m48s
redemption-api-57cfff9748-zxh9w     0/1     Terminating       0           5m48s
redemption-api-57cfff9748-7sh6s     1/1     Running            0           5m49s
redemption-api-57cfff9748-7sh6s     1/1     Terminating       0           5m49s
redemption-api-57cfff9748-7sh6s     1/1     Terminating       0           5m49s
redemption-api-57cfff9748-vgq8w     0/1     Pending            0           0s
redemption-api-57cfff9748-vgq8w     0/1     Pending            0           0s
redemption-api-57cfff9748-7sh6s     0/1     Terminating       0           5m49s
redemption-api-57cfff9748-7sh6s     0/1     Terminating       0           5m50s
redemption-api-57cfff9748-7sh6s     0/1     Terminating       0           5m50s
```

Above Figure is Validation of Kubernetes Pod Disruption Budgets (PDB) during a simulated worker node failure. The Eviction API strictly enforced PDB constraints by blocking the termination of critical system pods to prevent downtime, while the deployment controller instantly provisioned replacement application pods onto healthy nodes.

Day-2 Operations & GitOps

Immutable Infrastructure and the Single Source of Truth

To drastically reduce operational toil and eliminate the risk of configuration drift during Day-2 operations, The Redemption microservice strictly adheres to GitOps principles. The GitHub repository serves as the immutable, single source of truth for all infrastructure and application states. Direct manual modifications to the cluster via kubectl are administratively discouraged and actively reverted by the deployment pipeline.

Automated Reconciliation via ArgoCD

ArgoCD is deployed as the continuous delivery controller within the EKS cluster. It is configured with an automated sync policy equipped with self-healing and pruning capabilities.

- **Continuous Monitoring:** ArgoCD continuously polls the Git repository for changes to the Helm charts and Kubernetes manifests.
- **Self-Healing Automation:** If an unauthorized change is made directly to the live cluster (e.g., a manual image tag update or a replica count change not driven by the HPA), ArgoCD instantly detects the state mismatch. It automatically overrides the manual intervention, syncing the cluster back to the exact state defined in the Git repository. This guarantees auditability, rollback capability, and strict version control for all production deployments.

GitOps Validation and Evidence

To validate the self-healing capability of the GitOps pipeline, an unauthorized manual kubectl set image command was executed against the live deployment.

```
WaiYan@RGN24NB-dmcOVkp MINGW64 ~/Desktop/terraform/accor-redemption-eks-lab/my-gitops-infra (main)
• $ kubectl set image deploy/redemption-api redemption-api=nginx:alpine -n redemption
deployment.apps/redemption-api image updated

WaiYan@RGN24NB-dmcOVkp MINGW64 ~/Desktop/terraform/accor-redemption-eks-lab/my-gitops-infra (main)
• $ kubectl rollout status deploy/redemption-api -n redemption
deployment "redemption-api" successfully rolled out

WaiYan@RGN24NB-dmcOVkp MINGW64 ~/Desktop/terraform/accor-redemption-eks-lab/my-gitops-infra (main)
○ $ kubectl get pods -n redemption -w
NAME                                READY   STATUS    RESTARTS   AGE
redemption-api-57cfff9748-dpkxp     1/1     Running   0           24h
redemption-api-57cfff9748-qnc9d     1/1     Running   0           22h
```

Validation of ArgoCD self-healing capabilities. The controller successfully identified an unauthorized manual cluster modification and automatically synchronized the deployment back to the approved desired state defined in the Git repository.

Role Definitions and Responsibilities

Senior Cloud Engineer (Team Lead)

Focus: Core Architecture, Security, and Pipeline Bootstrapping.

Responsibilities:

- Design the overall AWS VPC topology, subnetting strategy, and routing (NAT/IGW).
- Provision the foundational AWS infrastructure using layered Terraform state files (Network, EKS Cluster, Database).
- Configure strict IAM Roles for Service Accounts (IRSA) and AWS Key Management Service (KMS) encryption policies.
- Bootstrap the ArgoCD controller into the EKS cluster and establish the initial Git repository connection.
- Conduct final architectural reviews and approve critical pull requests.

For Junior Cloud Engineer 1 (Application & Traffic Routing)

Focus: Kubernetes Workloads and Traffic Ingress.

Responsibilities:

- Develop and package the redemption-api application using Helm charts.
- Define the Kubernetes Deployment, Service, and Horizontal Pod Autoscaler (HPA) manifests.
- Configure the AWS Load Balancer Controller and define the Ingress rules for public traffic routing.
- Execute synthetic load testing (k6) to validate the HPA scaling thresholds.

For Junior Cloud Engineer 2 (Data & Observability)

Focus: State Management and Telemetry.

Responsibilities:

- Manage the AWS Secrets Manager configurations to securely inject database credentials into the cluster.
- Deploy and configure the kube-prometheus-stack (Prometheus and Grafana) via Helm for cluster telemetry.
- Verify database connectivity between the worker nodes and the Multi-AZ RDS PostgreSQL instance.
- Configure baseline alerting thresholds for CPU and memory exhaustion.

